SOLUTION ARCHITECTURE & ENGINEERING DOCUMENT

ProcureIQ
Enterprise Agentic FinTech & Procurement Intelligence Platform

System Design, AI Architecture & Production Engineering Reference

Document Type	Solution Architecture Document (SAD)
Version	1.0
Classification	Confidential — Internal / Interview Use
Platform	Multi-Tenant Agentic RAG System with HITL Governance
Tech Stack	FastAPI · LangGraph · PostgreSQL/pgvector · Qwen3-4B · Gemini
Test Coverage	31/31 Automated Tests Passing (Phases 1–5)

1. Executive Summary

ProcureIQ is a production-grade, multi-tenant agentic AI platform designed to automate enterprise FinTech and procurement workflows. The platform combines a stateful LangGraph orchestration engine, hybrid vector + full-text retrieval, role-based access control (RBAC), and human-in-the-loop (HITL) approval governance into a single cohesive architecture.

The core design challenge addressed by this system is **scalable tool-calling**: rather than injecting all API tool definitions into every LLM prompt (which grows linearly with registry size), ProcureIQ retrieves only the most relevant tools per query using the same retrieval algorithms applied to documents in a traditional RAG pipeline. This keeps LLM context size constant regardless of how many tools are registered.

O(k) constant	≥ 87.5% Recall@5	2560d HALFVEC	31 / 31
Tool binding complexity — 50 to 500+ tools, always top-k=5 retrieved	Verified at < 200ms latency on 500-tool benchmark	Qwen3-4B FP16 embeddings in PostgreSQL pgvector	Automated integration tests passing across all phases

2. System Architecture

The system is structured as a layered architecture with clearly separated concerns across transport security, cognitive orchestration, hybrid retrieval, policy enforcement, and durable persistence. Figure 1 illustrates the high-level component topology.

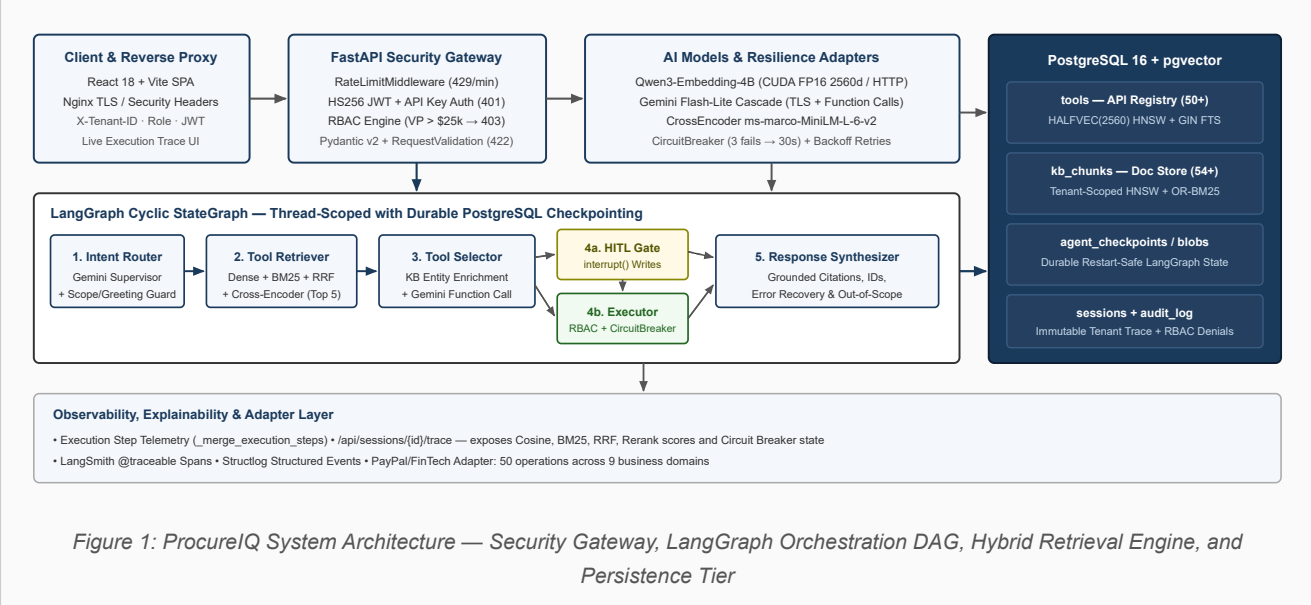


Figure 1: ProcureIQ System Architecture — Security Gateway, LangGraph Orchestration DAG, Hybrid Retrieval Engine, and Persistence Tier

3. Architectural Design Decisions

The following table documents the principal architectural choices made during the design of ProcureIQ, the rationale behind each, and the alternative approaches that were evaluated and rejected.

Design Decision	Rationale	Rejected Alternatives & Risk Avoided
LangGraph Cyclic StateGraph with interrupt()	Enables clean separation of intent routing, retrieval, enrichment, safety gating, execution, and synthesis into independently testable, deterministic graph nodes. Supports stateful pause/resume across HTTP turns for financial write operations requiring human approval.	Stateless ReAct loops cannot pause mid-execution for human confirmation. Single-shot chains cannot maintain multi-user RBAC handoff state across separate HTTP requests.
Custom asyncpg PostgresCheckpointSaver	Stores full graph snapshots, channel blobs, and pending writes in PostgreSQL via SQLAlchemy ON CONFLICT upserts. Shares the application's existing connection pool, eliminating a separate persistence service.	InMemorySaver loses all pending HITL approvals on server restart or multi-worker load balancing. The official psycopg3 async saver triggers Windows ProactorEventLoop conflicts and splits SQLAlchemy's connection pool.
Dynamic Tool RAG — Top-k Binding	Decouples tool registry growth from LLM context size. Only the top-5 most relevant tool schemas are injected per query, maintaining constant O(k) prompt token count regardless of whether 50 or 500 tools are registered.	Injecting all tool schemas into every system prompt causes O(N) token growth, exhausts context windows, and significantly reduces tool selection accuracy due to attention dilution on near-duplicate operations.
PostgreSQL as Unified Data Store	A single PostgreSQL 16 instance handles HNSW vector search, BM25 full-text retrieval, transactional sessions, durable checkpoints, and an immutable audit log — all within one ACID-compliant database with full tenant isolation via indexed tenant_id predicates.	Separate vector databases (Pinecone, Qdrant) + Redis + PostgreSQL increases operational complexity, introduces eventual-consistency issues on record deletes, and complicates tenant isolation boundary enforcement.
Pre-Selection Knowledge Base Enrichment	Entity identifiers (invoice IDs, PO numbers, emails, amounts) are resolved from the tenant's knowledge base and prior audit logs before the LLM selects and parameterises a tool. This ensures that all required parameters are available at function-call time.	Requiring the LLM to call a lookup tool first to retrieve missing parameters creates multi-turn ambiguity, risks incorrect parameter binding, and fails HITL pre-validation for financial operations.

3.1 End-to-End Request Lifecycle

The following walkthrough traces a representative high-risk request ("Approve purchase order PO-2026-0301 for $32,000") through each system component to illustrate how security, retrieval, durable state, and RBAC enforcement interact:

Step 1 — Gateway & Authentication

The rate limiter validates the sliding-window IP quota. `get_auth_context` resolves tenant_id and RBAC role (operator) from the HS256 JWT token and enforces session ownership.

Step 2 — Intent Classification & Tool Retrieval

The intent classifier routes to the *procurement* domain. The tool retriever embeds the query via Qwen3-4B GPU, executes HNSW + GIN OR-BM25 + RRF + Cross-Encoder reranking across 50 registered tools, and binds the top-5 candidates.

Step 3 — KB Enrichment & Tool Selection

`_enrich_params_from_kb` resolves the PO document (vendor: Aegis Cyber, amount: $32,000). Gemini's native functionCall mechanism selects `approve_purchase_order`.

Step 4 — Durable HITL Pause

As a registered write-class operation, `hitl_gate` calls `interrupt()`, persisting the full graph state (channel blobs + pending writes) to PostgreSQL. The state survives server restarts.

Step 5 — RBAC Policy Enforcement

If an operator attempts to confirm, `enforce_tool_rbac` returns HTTP 403, citing the $25,000 Finance VP approval threshold, while leaving the checkpoint intact for a qualified approver.

Step 6 — VP Approval & Audit

A finance_vp resumes the session via `/api/chat/confirm`. The approver_role is injected into the resumed state; the executor runs behind a CircuitBreaker and writes a full audit trace to audit_log.

3.2 Enterprise Domain Coverage

The tool registry covers **9 capability domains** across 50 registered API operations: Payments & Treasury (captures, payouts, refunds, subscriptions), Invoicing & 3-Way Match Discrepancy Resolution, Procurement (purchase orders, goods receipts, RFQs, vendor onboarding), Disputes & Risk/KYC, Financial Analytics, Webhook Management, Knowledge Base RAG (54 indexed documents in TXT, MD, PDF, and DOCX formats including contracts, SLAs, budget reports, SOC2 and FINRA governance SOPs), and System Introspection.

4. AI Models & Retrieval Algorithms

4.1 Model Stack

Embedding Model — Qwen3-Embedding-4B

Executed natively on CUDA GPU in FP16 half-precision via HuggingFace Transformers (device_map auto, 6,800 MiB VRAM budget). Uses causal **last-token pooling** followed by L2 unit normalisation to produce 2,560-dimensional sentence embeddings. Supports an HTTP-first mode that redirects embedding requests to a shared vLLM/Ollama/TEI endpoint, enabling multi-worker deployments without duplicating GPU VRAM per worker process.

Reasoning & Tool Calling — Google Gemini

An ordered resilience cascade targets gemini-2.5-flash-lite → flash-lite-latest → flash-latest with TLS verification enforced. Native OpenAPI `functionDeclarations` (mode: ANY) are issued against the top-5 bound tools plus KB context window. A deterministic regex/heuristic fallback activates on 429 quota throttle responses, ensuring the system degrades gracefully without returning errors to end users.

Reranker — Cross-Encoder (ms-marco-MiniLM-L-6-v2)

After the initial hybrid retrieval pass, the top-15 candidates are re-scored using a cross-encoder cross-attention model. Scores are normalised via sigmoid to produce a [0,1] relevance probability that feeds into the composite final scoring formula. A zero-download lexical concept alignment fallback is used if the model is unavailable.

4.2 HALFVEC(2560) — Design Rationale

Standard open-source pgvector limits HNSW index construction on single-precision `vector` columns to a maximum of 2,000 dimensions. Qwen3-Embedding-4B outputs 2,560-dimensional vectors, exceeding this constraint. The solution uses PostgreSQL's `halfvec` type (FP16 storage), which supports HNSW indexing up to 4,000 dimensions using `halfvec_cosine_ops`. This yields full 2,560d semantic fidelity while reducing HNSW index memory consumption by approximately 50%.

A dimension alignment utility (`align_embedding_dimension`) applies orthogonal projection and L2 re-normalisation for any fallback embedding provider (768d BGE, 384d hash, 1536d OpenAI, 3072d Gemini), ensuring vector tables remain schema-stable across provider changes without DDL migrations.

4.3 Hybrid Retrieval Pipeline — Four-Stage Algorithm

The same four-stage pipeline is used for both the Knowledge Base (HybridSearchService, 54+ documents) and Dynamic Tool RAG (ToolRetriever, 50–500+ tools):

Stage 1 **Sparse Search**	**Query normalisation + OR-BM25 full-text search.** Typos are corrected and stopwords are stripped. Query tokens are joined with OR operators so conversational filler words do not cause zero-result BM25 matches. Executed via PostgreSQL GIN-indexed `to_tsvector` with `ts_rank_cd` scoring. `ts_q = " | ".join(tokens(normalize_query(q))) → ts_rank_cd(to_tsvector(doc), to_tsquery(ts_q))`
Stage 2 **Dense Search**	**pgvector HNSW approximate nearest-neighbour search** over HALFVEC(2560) embeddings. Sub-linear query complexity; cosine similarity is normalised to [0.0, 1.0]. `cos(q, d) = max(0.0, 1.0 − (emb(d) <=> emb(q)))`
Stage 3 **Rank Fusion**	**Reciprocal Rank Fusion (RRF)** merges sparse and dense result lists in a scale-invariant manner, without requiring score calibration or min-max normalisation between the two retrieval modalities. `RRF(d) = 1 / (60 + rank_dense(d)) + 1 / (60 + rank_bm25(d))`
Stage 4 **Final Scoring**	**Composite score with Intent Safety Multiplier.** Blends RRF, Cross-Encoder relevance, and cosine similarity. Destructive tools (cancel, void, delete) are penalised on lookup-intent queries via the M_intent multiplier to prevent write-on-read hallucinations. Values: 0.55× for destructive verbs, 0.65× for mutating writes on lookup cues, 1.0× otherwise. `S_final(t) = [ 0.45·(RRF/RRF_max) + 0.35·sigmoid(CrossEncoder(q, t)) + 0.20·cos(q, t) ] × M_intent(q, t)`

Design note — OR-BM25 and M_intent in financial contexts: Standard dense retrieval ranks *cancel_invoice* and *get_invoice* nearly identically for a query such as "get me the Mike invoice already sent", since the semantic vectors overlap strongly on the shared token "invoice". The combination of OR-BM25 (for exact identifier matching), RRF (rank fusion), Cross-Encoder cross-attention, and the intent safety multiplier ensures that read-intent queries consistently bind read-only tools while write-intent operations route through HITL confirmation and RBAC enforcement.

5. Scalability & Fault Tolerance

5.1 Scalability Design

Horizontal Scalability

- **Sub-linear index growth:** PostgreSQL HNSW graphs and GIN inverted indexes maintain sub-200ms retrieval latency as the tool registry scales from 50 to 500+ entries, verified via the 500-tool benchmark test.
- **Stateless API workers:** Storing all LangGraph state and HITL interrupts in PostgreSQL means any Uvicorn worker behind Nginx or an ALB can resume a paused workflow. Zero session affinity is required, enabling standard rolling deployments.
- **Shared embedding endpoint:** Setting `QWEN_USE_HTTP_FIRST=true` routes all backend workers to a shared HTTP inference server (vLLM, Ollama, or TEI), eliminating the N × 6.5 GB VRAM overhead of loading the embedding model per worker process.

Performance Benchmarks

Benchmark	Measured Result
500-Tool Scale Test (Recall@5)	**≥ 87.5%** at < 200 ms
Live Top-1 Accuracy (6 domain probes, 50 tools)	**6/6 (100%)**, mean 421 ms warm
54-Document KB Ingestion (TXT/MD/PDF/DOCX)	**54/54** docs, 124.57s (~2.3s/doc GPU)
Durable HITL Restart Test (PO $32k)	Survived process kill + reboot, 200 OK

Connection pooling: FastAPI + asyncpg + SQLAlchemy async pool (pool_size=10, max_overflow=20) prevents event-loop starvation while offloading CUDA tensor operations to executor threads.

5.2 Resilience — Circuit Breaker State Machine

A per-adapter circuit breaker (implemented in resilience.py) wraps each external integration (PayPal API, RAG service, system search). It transitions through three states to prevent cascading failures:



Figure 2: Circuit Breaker State Machine and Observability Architecture

6. Security & Input Validation

Control Area	Implementation	Threat / Failure Mitigated
Authentication & RBAC	Timing-safe HMAC-SHA256 JWT (`create_access_token`, `decode_access_token`) and API key CSV support. Four-tier RBAC: viewer, operator, finance_vp, admin. The finance_vp tier is required for approve_purchase_order operations exceeding $25,000, enforced per Vendor Onboarding Policy. The approver_role is propagated into resumed checkpoint state to prevent role downgrade on HITL resumption.	Unauthorised API access (401), privilege escalation, operator self-approval of capital expenditure purchase orders (403 + rbac_denied audit entry).
Multi-Tenant Isolation	An indexed `tenant_id` column is applied as a mandatory SQL predicate in all HNSW/BM25 retrieval queries, KB CRUD operations, session state access, trace retrieval, and confirmation endpoints. Cross-tenant session ownership is enforced with 403 responses.	Cross-tenant data exposure in RAG prompts or unauthorised session access.
Input & Scope Validation	A three-layer scope guard (intent_classifier rules + Gemini supervisor + tool_retriever semantic score floor) handles off-topic queries (weather, trivia, general conversation) with a helpful out-of-scope response (tool_used=null), never executing tools. Pydantic v2 validates message length, session_id regex, positive-only confirmation amounts, file extensions, and 25 MB upload cap with human-readable 422 error strings.	Off-topic tool hallucinations, SQL truncation, binary/empty file uploads, negative payment overrides, UI serialisation errors.
Rate Limiting & Transport	Per-IP 60-second sliding-window RateLimitMiddleware returns 429 with Retry-After: 60. Gemini API TLS verification is enforced by default (GEMINI_VERIFY_SSL=true). Nginx is configured with security headers: X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, Referrer-Policy: strict-origin-when-cross-origin.	Brute-force API abuse, quota exhaustion via high-frequency clients, unverified upstream TLS connections.

7. Technology Stack & Verification

Architecture Layer	Technologies & Components
Frontend & Reverse Proxy	React 18, Vite 6, Lucide Icons. Multi-stage Docker build produces a static bundle served by nginx:1.27-alpine with a 25 MB upload limit, 120-second proxy timeout, and workspace-scoped tenant/role/JWT controls in SettingsPanel.jsx.
API & Security Gateway	FastAPI 0.115, Uvicorn (non-reload production mode via entrypoint.sh), Pydantic v2, custom HS256 JWT + API key authentication, four-tier RBAC engine, and sliding-window RateLimitMiddleware (app/core/security.py).
Agent Orchestration	LangGraph 0.2.60 with a custom PostgresCheckpointSaver (BaseCheckpointSaver), cyclic StateGraph with interrupt()-based HITL nodes, structured execution step telemetry via _merge_execution_steps reducer, and LangSmith @traceable span instrumentation.
AI & Retrieval Models	Qwen/Qwen3-Embedding-4B (HuggingFace Transformers, PyTorch FP16 CUDA), Google Gemini REST API (native functionDeclarations, ordered resilience cascade), CrossEncoder ms-marco-MiniLM-L-6-v2 (sentence-transformers), PyMuPDF (PDF parsing), python-docx (DOCX parsing).
Data & Storage	PostgreSQL 16 + pgvector extension: HALFVEC(2560) HNSW index (halfvec_cosine_ops) + GIN full-text index (OR-BM25). SQLAlchemy 2.0 async core + asyncpg driver. Alembic migrations (001: schema init, 002: HALFVEC/GIN upgrade). Tables: tools, kb_chunks, sessions, agent_checkpoints, agent_checkpoint_blobs, agent_checkpoint_writes, audit_log.
Infrastructure & Packaging	Docker Compose full-stack orchestration (db + backend + frontend services). Backend Dockerfile with multi-stage build. Playwright Chromium for document generation. Nginx configuration with security headers and proxy settings.
Automated Testing	**31/31 pytest integration tests passing** across five phases: Phase 1 — Tool RAG retrieval accuracy; Phase 2 — LangGraph HITL interrupt/resume; Phase 3 — Circuit Breaker state transitions + 500-tool scale benchmark; Phase 4 — Gemini function calling + KB entity disambiguation; Phase 5 — Durable checkpoint recovery, RBAC 403 enforcement, multi-tenant isolation, JWT 401, rate limit 429, out-of-scope guard, and Pydantic 422 validation. Additional: clean Vite production build and Docker Compose YAML validation.

8. Known Constraints & Future Enhancements

Area	Current Constraint / Planned Enhancement
PayPal Live API Integration	The PayPal adapter is implemented in full (50 operations, sandbox-verified tool schemas, HITL and RBAC enforcement). Live production credential integration is excluded from the current scope; the adapter accepts live credentials via environment variables when available.
Embedding Worker Isolation	In the current single-node configuration, Qwen3-4B loads once per worker process. For multi-worker production deployments, setting QWEN_USE_HTTP_FIRST=true is required to route all workers to a shared inference endpoint and prevent GPU VRAM exhaustion.
Streaming Response Support	The current API returns complete responses after graph execution. Server-Sent Events (SSE) streaming is architecturally compatible with LangGraph's astream_events interface and is planned for a subsequent release.
Fine-Tuning & Domain Adaptation	The current system uses off-the-shelf Qwen3-Embedding-4B without domain-specific fine-tuning. LoRA adapter training on enterprise FinTech query-document pairs is planned to further improve retrieval precision for highly specialised financial terminology.